DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

Submitted by	
Name	Namish Pande
Roll No	**AV.SC.U4CSE25044**
Year/Sem/Section	**2nd year/Sem-3/CSE-A**
Credits	**05**

Submitted to	
Name	**DR. RAJ KUMAR BATCHU**
Department	**DEPARTMENT OF COMPUTER SCIENCE**
Designation	**ASSISTANT PROFESSOR**

Table of Contents

S. No	*Title*	*Page No*
1	Recursion	1-5
2	Binary Search and Linear Search	5-8
3	Sorting Methods	8-12
4	Linked Lists	13-27
5	Stack	28-34
6	Queues and Circular Queues	35-45
7	Binary Trees	46-56
8	Binary Search Trees	56-60
9	AVL Trees	60-66
10	Hashing	67-72

Week 1

Aim: To understand and implement recursion in problem solving.

1. A rocket launch system displays a countdown before the launch. Write a recursive program to display from n to 1, followed by launch.

```
## 1.

def countdown(n):
    if n == 0:
        print("Launch!")
    else:
        print(n)
        countdown(n-1)

countdown(10)
```

Output:

```
PS C:\Users\CSE-Lab1\25044\DSA> python3 .\03-LAB.py
10
9
8
7
6
5
4
3
2
1
Launch!
```

Explanation:

This is a basic recursion loop. It starts with 10, prints the number and calls the function again with n = n-1. If n == 0, it only prints “Launch” and the function terminates.

Error Table: nil

2. Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a program to calculate P^(n) where: P = principal growth factor, n = number of years

```
### 2.
p = float(input("Enter principal amount: "))
n = int(input("Enter years: "))

def calculate(p, n):
    if n == 1:
        return p
    return p * calculate(p, n-1)

print(calculate(p,n))
```

Output:

```
TERMINAL

PS C:\Users\CSE-Lab1\25044\DSA> python3 .\03-LAB.py
Enter principal amount: 100
Enter years: 4
100000000.0
PS C:\Users\CSE-Lab1\25044\DSA>
```

Explanation: The function calculate takes in two arguments: principal amount and number of years. Since we are calculating p^n, we just return p * p^(n-1) repeated until n == 1, by which point we return p itself, since p^1 == p.

- float() – for typecasting input to float
- int() – typecasting input to int
- input() – taking in user input

Error table: nil

3. An hr department stores ids in a list. develop a recursive program to search for a given employee id, write a recursive function to search form an employee id in a list.

```python
### 3.
emp_data = [input(f"Enter Id of Employee {i+1}: ") for i in range(int(input("Number of employees: ")))]
def search_for_emp(li, id, n):
    if li[n] == id:
        print(f"id found at index: {n}")
        return

    if n >= len(li) - 1:
        print(f"id not found: {n}")
        return

    return search_for_emp(li, id, n+1)
to_search = input("id to search for: ")
search_for_emp(emp_data, to_search, 0)
```

Output:

```
TERMINAL

PS C:\Users\CSE-Lab1\25044\DSA> python3 .\03-LAB.py
Number of employees: 5
Enter Id of Employee 1: a
Enter Id of Employee 2: b
Enter Id of Employee 3: c
Enter Id of Employee 4: d
Enter Id of Employee 5: e
id to search for: e
id found at index: 4
PS C:\Users\CSE-Lab1\25044\DSA>
```

Explanation: We start by leveraging list comprehension to and inputs to take in all the employee data in a one liner. Then, the search_for_emp function takes in 3 arguments, the emp list itself, the id to check for and the index, which we will pass in as 0.

If the is is found at that index, we print success and return, else if, we have exceeded the bounds of the list, we say that id was not found, or else, we just move on forward and call the recursive function with the argument n+1.

[I for I in X] – list comprehension syntax

- input() – get user input
- range() – generates an iterable sequence

Error table: nil

4. In a logistics company, the number of possible way to arrange n different parcels is calculated using the factorial formula. (n!) develop a recursive program to calculate the factorial of a given number.

```
### 4.

def factorial(f):
    if f == 0 or f == 1:
        return 1
    return f * factorial(f-1)

print(factorial(int(input())))

```

Output:

\

Output

Explanation: This code is for taking the user input and displaying its factorial. We know mathematically that n! = n x (n-1)!

The factorial function only takes one argument which is N and is recursively multiplies itself with factorial(N-1) until it reaches 1 or 0, in which case it just returns 1.

- int() – typecasting to integer
- input() – getting user input

Error table: nil

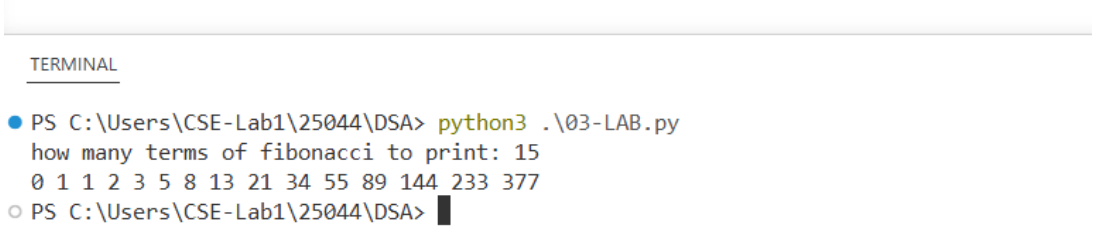
5. Write a recursive python program to print the first n terms of Fibonacci series.

Output:

Explanation: the Fibonacci function only takes in one argument and returns the nth Fibonacci number. The base cases are there for the first two items in the series, after which, the nth terms becomes the sum of the previous two terms.

After that, we take a separate for loop where we loop over to till we have to print the series and print the nth Fibonacci 1 by 1.

Error table: nil



Week 2

Aim: To learn about the various searching methods

1. WAP to perform linear search on the given user input

05-LAB-SEARCH.py > ...

```python
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i

    return -1


a = [int(input(f"Entry #{i+1}: ")) for i in range(int(input("How many entries do you want to add: ")))]
x = int(input("item to find: "))
i = linear_search(a, x)
print(f"item found at index {i}" if  i >= 0 else "item not found")
```

O

```
PS C:\Users\CSE-Lab1\25044\DSA> python .\05-LAB-SEARCH.py
How many entries do you want to add: 5
Entry #1: 2
Entry #2: 4
Entry #3: 1
Entry #4: 42
Entry #5: 32
item to find: 32
item found at index 4
PS C:\Users\CSE-Lab1\25044\DSA>
```

Explanation: Linear search is the simplest searching algorithm that works on all kinds of unsorted/sorted data. It loops through a list one by one and returns the index if it's found, else it returns –1.

Error Table: nil

2. WAP to perform binary search on the given input to perform binary search on the given input.

01-LAB.py | 04-LAB.py | 05-LAB-SEARCH.py | 02-LAB.py | 03-LAB.py

05-LAB-SEARCH.py > ...

```python
def binary(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1


a = [int(input(f"Entry #{i+1}: ")) for i in range(int(input("How many entries do you want to add: ")))]
x = int(input("item to find: "))
i = binary(a, x)
print(f"item found at index {i}" if  i >= 0 else "item not found")
```

Output:

1. Case 1: sorted input

```
PS C:\Users\CSE-Lab1\25044\DSA> python .\05-LAB-SEARCH.py
How many entries do you want to add: 7
Entry #1: 1
Entry #2: 4
Entry #3: 6
Entry #4: 8
Entry #5: 9
Entry #6: 12
Entry #7: 16
item to find: 9
item found at index 4
```

2. Case 2: unsorted input

```
PS C:\Users\CSE-Lab1\25044\DSA> python .\05-LAB-SEARCH.py
How many entries do you want to add: 5
Entry #1: 12
Entry #2: 2
Entry #3: 67
Entry #4: 34
Entry #5: 9
item to find: 2
item not found
```

Explanation: As we can see in the above example, the algorithm only works for sorted inputs, since it helps it to effectively divide the search range by half every time.

Week 3

Aim: To learn about the various sorting algorithms.

1. Write a program to sort the elements using bubble sort technique. Take input from users.

bubble.py

week_3 > bubble.py > bubble

```python
def bubble(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break

    return arr

a = [int(input(f"Entry #{i+1}: ")) for i in range(int(input("How many entries do you want to add: ")))]
print(bubble(a))
```

Output

```
PS C:\Users\CSE-Lab1\25044\DSA\week_3> python .\bubble.py
How many entries do you want to add: 5
Entry #1: 1232
Entry #2: 23
Entry #3: 115
Entry #4: 887
Entry #5: 45
[23, 45, 115, 887, 1232]
PS C:\Users\CSE-Lab1\25044\DSA\week_3> S
```

Explanation: This code uses bubble sort to arrange a list of numbers from smallest to largest. The bubble() function first finds the number of elements in the list and then repeatedly compares two neighboring elements. If the left element is bigger than the right one, they are swapped, causing the larger numbers to gradually move toward the end of the list. After each pass, the program checks whether any swapping happened using the swapped variable; if nothing was swapped, it means the list is already sorted, so the loop stops early. The return arr then gives back the sorted list. The last two lines ask the user how many numbers they want to enter, take each number as input, store them in a list, pass that list to the bubble() function, and finally print the sorted result.

Error Table: nil

2. Write a program to sort the elements using insertion sort technique. Take input from users.

inserstion.py

week_3 > inserstion.py > ...

```python
def isort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr


a = [int(input(f"Entry #{i+1}: ")) for i in range(int(input("How many entries do you want to add: ")))]
print(isort(a))
```

Output:



Explanation: This code uses insertion sort to arrange a list of numbers from smallest to largest. The isort() function starts from the second element and treats the elements before it as an already sorted part of the list. It stores the current element in key and compares it with the elements before it; if an earlier element is bigger than key, it shifts that element one position to the right to make space. This continues until the correct position for key is found, and then key is placed there. The process repeats for every element until the entire list is sorted. The last two lines ask the user how many numbers they want to enter, take those numbers as input, store them in a list, pass the list to isort(), and print the sorted result.

Error table: nil

3. Write a program to sort the elements using selection sort technique. Take input from users.

selection.py

week_3 > selection.py > ...

```python
def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr

a = [int(input(f"Entry #{i+1}: ")) for i in range(int(input("How many entries do you want to add: ")))]
print(selection_sort(a))
```

Output:

```
PS C:\Users\CSE-Lab1\25044\DSA\week_3> python .\selection.py
How many entries do you want to add: 6
Entry #1: 423
Entry #2: 41
Entry #3: 13
Entry #4: 351
Entry #5: 342
Entry #6: 10324
[13, 41, 342, 351, 423, 10324]
PS C:\Users\CSE-Lab1\25044\DSA\week_3> S
```

Explanation: This code uses selection sort to arrange a list of numbers from smallest to largest. The selection_sort() function goes through the list and, for each position, searches the remaining unsorted part to find the smallest number. It stores the position of this smallest number in min_index and, once the search is complete, swaps that number with the element at the current position. This process continues until all the numbers are in the correct order. The last two lines ask the user how many numbers they want to enter, take each number as input, store them in a list, pass the list to selection_sort(), and print the sorted result.

Error table: nil

4. Write a program to sort the elements using merge sort technique. Take input from users.

```python
def merge(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge(arr[:mid])
    right = merge(arr[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

a = [int(input(f"Entry #{i+1}: ")) for i in range(int(input("How many entries do you want to add: ")))]
print(merge(a))
```

Output:

```
PS C:\Users\CSE-Lab1\25044\DSA\week_3> python .\merge.py
How many entries do you want to add: 5
Entry #1: 354
Entry #2: 342
Entry #3: 23
Entry #4: 12
Entry #5: 4536
[12, 23, 342, 354, 4536]
```

Explanation: This code uses merge sort to arrange a list of numbers from smallest to largest. The merge() function first divides the list into two smaller halves repeatedly until each part contains only one element, since a single element is already sorted. It then compares the elements of the left and right halves, picks the smaller one, and adds it to result. Once one half is completely used, the remaining elements from the other half are added to result. This merging process continues until all the smaller lists are combined back into one sorted list. The last two lines ask the user how many numbers they want to enter, take the numbers as input, store them in a list, pass the list to merge(), and print the sorted result.

Error table: nil.

5. Write a program to sort the elements using quick sort technique. Take input from users.

merge.py | quick.py

week_3 > quick.py > ...

```python
def quick(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[-1]

    left = []
    right = []

    for x in arr[:-1]:
        if x <= pivot:
            left.append(x)
        else:
            right.append(x)

    return quick(left) + [pivot] + quick(right)


a = [int(input(f"Entry #{i+1}: ")) for i in range(int(input("How many entries do you want to add: ")))]
print(quick(a))
```

Output:

```
PS C:\Users\CSE-Lab1\25044\DSA\week_3> python .\quick.py
How many entries do you want to add: 5
Entry #1: 4532
Entry #2: 34
Entry #3: 1
Entry #4: 98
Entry #5: 9845
[1, 34, 98, 4532, 9845]
PS C:\Users\CSE-Lab1\25044\DSA\week_3>
```

Explanation:

This code uses quick sort to arrange a list of numbers from smallest to largest. The quick() function first checks if the list has one or no elements, in which case it is already sorted. Otherwise, it chooses the last

element as the pivot and compares every other element with it. Numbers smaller than or equal to the pivot are placed in the left list, while larger numbers are placed in the right list. The function then recursively sorts both the left and right lists and combines them with the pivot in the middle, giving the final sorted list. The last two lines ask the user how many numbers they want to enter, take the numbers as input, store them in a list, pass the list to quick(), and print the sorted result.

Error table: nil

Week 4

1. Write a python program to implement a singly linked list which could perform following operations by taking an input from the user, creating an linked list, insert in beginning, insert at end, insert at index, delete by value, delete frist node, delete last node, count number of nodes, display or traverse the list, exit.

```
class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class LinkedList:
```

```
def __init__(self):
    self.hea class d = None
    self.capacity = 0

def shift(self, data):
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node
    self.capacity += 1

def append(self, data):
    new_node = Node(data)
    if not self.head:
        self.head = new_node
        self.capacity += 1
        return

    curr = self.head
    while curr.next:
        curr = curr.next
    curr.next = new_node
    self.capacity += 1

def insert(self, data, n):
    if n > self.capacity:
        print("Error: index out of bounds")
        return
    if n < 0:
        print("Negative Index")
        return

    new_node = Node(data)
    curr = self.head
    pos = 0
    while curr and pos < n - 1:
        curr = curr.next

        pos += 1

    new_node.next = curr.next
    curr.next = new_node
    self.capacity += 1

def remove_start(self):
    if not self.head:
        return
    self.head = self.head.next
    self.capacity -= 1
```

```
def remove_end(self):
    if not self.head:
        return

    curr = self.head
    while curr.next.next:
        curr = curr.next

    curr.next = None

    self.capacity -= 1

def remove_by_value(self, n):
    if not self.head:
        return
    curr = self.head

    if curr and curr.data == n:
        self.head = curr.next
        self.capacity -= 1
        return

    prev = None
    while curr and curr.data != n:
        prev = curr
        curr = curr.next

    if not curr:
        print("not found")
        return

    prev.next = curr.next
    curr = None

    self.capacity -= 1


def __str__(self):
    current = self.head
    str = ''
    while current:
        str += f'{current.data} '
        current = current.next
    return str.rstrip()


def __len__(self):
    return self.capacity
```

```
ll = None

menu="'1.Create Linked List\n2.Insert at beginning\n3.Insert at end\n4.Insert at nth index\n5.Remove from start\n6.Remove from end\n7.Remove by value\n8.Length\n9.Display\nQuit by entering anything else'"

while True:
    print(menu)
    s = input("Your choice:")
    if s == '1':
        ll = LinkedList()
        n = int(input("How many items do you want to enter: "))
        for i in range(n):
            x = eval(input(f'What value to insert at index {i}: '))
            ll.append(x)
    elif s == '2':
        x = eval(input("Item to insert: "))
        ll.shift(x)
    elif s == '3':
        x = eval(input("Item to insert: "))
        ll.append(x)
    elif s == '4':
        x = eval(input("Item to insert: "))
        y = int(input(f'Index to insert at (0 to {ll.capacity}): '))
        ll.insert(x, y)
    elif s == '5':
        ll.remove_start()
    elif s == '6':
        ll.remove_end()
    elif s == '7':
        x = eval(input("what value do you want removed: "))
        ll.remove_by_value(x)
    elif s == '8':
        print(len(ll))
    elif s == '9':
        print(ll)
    else:
        break
```

Output:

```
PS C:\Users\CSE-Lab1\25044\DSA\week_4> python3 .\singlylinkedlist.py
1.Create Linked List
2.Insert at beginning
3.Insert at end
4.Insert at nth index
5.Remove from start
6.Remove from end
7.Remove by value
8.Length
9.Display
Quit by entering anything else
Your choice:1
How many items do you want to enter: 4
What value to insert at index 0: 2
What value to insert at index 1: 423
What value to insert at index 2: 21
What value to insert at index 3: 43
1.Create Linked List
2.Insert at beginning
3.Insert at end
4.Insert at nth index
5.Remove from start
6.Remove from end
7.Remove by value
8.Length
9.Display
Quit by entering anything else
```

```
8.Length
9.Display
Quit by entering anything else
Your choice:2
Item to insert: 1
1.Create Linked List
2.Insert at beginning
3.Insert at end
4.Insert at nth index
5.Remove from start
6.Remove from end
7.Remove by value
8.Length
9.Display
Quit by entering anything else
Your choice:9
1 2 423 21 43
1.Create Linked List
2.Insert at beginning
3.Insert at end
4.Insert at nth index
5.Remove from start
6.Remove from end
7.Remove by value
8.Length
9.Display
```

Explanation: The following code demonstrates a Singly Linked List in python. There are two classes which are created: Node and LinkedList. Node is the most basic unit that exists inside a linked list and contains two attributes, the data it holds and the next node it points to. The LinkedLists only attribute is the head node and the count, which is a way to track the length.

The following functions are implemented inside the LinkedList:

shift(data): This function inserts a new node at the beginning of the linked list. The new node is made the new head, and its next attribute points to the previous head. The capacity is then increased by one.

append(data): This function inserts a new node at the end of the linked list. If the linked list is empty, the new node becomes the head. Otherwise, the function traverses the list until it reaches the last node and attaches the new node to it. The capacity is increased by one.

insert(data, n): This function inserts a new node at a specific index in the linked list. First, it checks if the index is within the valid range and is not negative. It then traverses the list until it reaches the node before the desired position and changes the pointers so that the new node is inserted. The capacity is increased by one.

remove_start(): This function removes the first node from the linked list. It does this by moving the head to the second node in the list. If the list is empty, nothing happens. The capacity is decreased by one.

remove_end(): This function removes the last node from the linked list. It traverses the list until it reaches the second-to-last node, then changes its next pointer to None. The capacity is decreased by one.

remove_by_value(n): This function searches for a node containing the given value and removes it from the linked list. If the value is found in the head node, the head is moved to the next node. Otherwise, the function keeps track of both the current and previous nodes so that the previous node can skip over the node being removed. If the value is not found, it prints "not found".

__str__(): This function determines how the linked list should be represented when it is printed. It traverses the list from the head and adds each node's data to a string, with each value separated by a space.

__len__(): This function allows the built-in len() function to be used on the linked list. It simply returns the capacity.

2. Write a python program to implement a doubly linked list which could perform following operations by taking an input from the user, creating an doubly linked list, insert in beginning, insert at end, insert at index, delete by value, delete frist node, delete last node, count number of nodes, display or traverse the list, exit.

```
class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None
        self.prev = None



class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.capacity = 0

    def shift(self, data):
        new_node = Node(data)
        new_node.next = self.head

        if self.head:
                self.head.prev = new_node

        self.head = new_node
        self.capacity += 1


    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            self.capacity += 1
            return

        curr = self.head
        while curr.next:
            curr = curr.next
```

```
        curr.next = new_node
        new_node.prev = curr
        self.capacity += 1

    def insert(self, data, n):
        if n > self.capacity:
            print("Error: index out of bounds")
            return
        if n < 0:
            print("Negative Index")
            return

        if n == 0:
            self.shift(data)
            return

        if n == self.capacity:
            self.append(data)
            return

        new_node = Node(data)
        curr = self.head
        pos = 0
        while curr and pos < n:
            curr = curr.next

            pos += 1

        new_node.next = curr
        new_node.prev = curr.prev

        curr.prev.next = new_node
        curr.prev = new_node

        self.capacity += 1

    def remove_start(self):
        if not self.head:
            return
        self.head = self.head.next
        self.capacity -= 1

        if self.head:
            self.head.prev = None


    def remove_end(self):
        if not self.head:
            return
```

```
        curr = self.head

        while curr.next:
            curr = curr.next

        if curr.prev:
            curr.prev.next = None
        else:
            self.head = None

        self.capacity -= 1

    def remove_by_value(self, n):
        if not self.head:
            return
        curr = self.head

        while curr.next and curr.data != n:
            curr = curr.next

        if not curr:
            print('Element to be removed not fuond')
            return

        if curr.prev is None:
            self.head = curr.next

            if self.head:
                self.head.prev = None

        else:
            curr.prev.next = curr.next
            if curr.next:
                curr.next.prev = curr.prev

        self.capacity -= 1

    def __str__(self):
        current = self.head
        str = ''
        while current:
            str += f'{current.data} '
            current = current.next
        return str.rstrip()


    def __len__(self):
        return self.capacity
```

```
dll = None

menu="'1.Create Doubly Linked List\n2.Insert at beginning\n3.Insert at end\n4.Insert at nth index\n5.Remove from start\n6.Remove from end\n7.Remove by value\n8.Length\n9.Display\nQuit by entering anything else'"

while True:
    print(menu)
    s = input("Your choice:")
    if s == '1':
        dll = DoublyLinkedList()
        n = int(input("How many items do you want to enter: "))
        for i in range(n):
            x = eval(input(f'What value to insert at index {i}: '))
            dll.append(x)
    elif s == '2':
        x = eval(input("Item to insert: "))
        dll.shift(x)
    elif s == '3':
        x = eval(input("Item to insert: "))
        dll.append(x)
    elif s == '4':
        x = eval(input("Item to insert: "))
        y = int(input(f'Index to insert at (0 to {dll.capacity}): '))
        dll.insert(x, y)
    elif s == '5':
        dll.remove_start()
    elif s == '6':
        dll.remove_end()
    elif s == '7':
        x = eval(input("what value do you want removed: "))
        dll.remove_by_value(x)
    elif s == '8':
        print(len(dll))
    elif s == '9':
        print(dll)
    else:
        break
```

Output:

```
PS C:\Users\CSE-Lab1\25044\DSA\week_4> python .\doublylinkedlist.py
1.Create Doubly Linked List
2.Insert at beginning
3.Insert at end
4.Insert at nth index
5.Remove from start
6.Remove from end
7.Remove by value
8.Length
9.Display
Quit by entering anything else
Your choice:1
How many items do you want to enter: 1
What value to insert at index 0: 23
1.Create Doubly Linked List
2.Insert at beginning
3.Insert at end
4.Insert at nth index
5.Remove from start
6.Remove from end
7.Remove by value
8.Length
9.Display
Quit by entering anything else
Your choice:2
Item to insert: 32
1.Create Doubly Linked List
2.Insert at beginning
3.Insert at end
4.Insert at nth index
5.Remove from start
6.Remove from end
7.Remove by value
8.Length
9.Display
Quit by entering anything else
Your choice:6
1.Create Doubly Linked List
2.Insert at beginning
3.Insert at end
4.Insert at nth index
5.Remove from start
6.Remove from end
7.Remove by value
8.Length
9.Display
Quit by entering anything else
Your choice:9
32
1.Create Doubly Linked List
```

Explanation: The following code demonstrates a Doubly Linked List in python. There are two classes which are created: Node and DoublyLinkedList. Node is the most basic unit that exists inside a linked list and contains three attributes, the data it holds, the next node it points to, and the previous node it points to. The DoublyLinkedList's only attributes are the head node and the capacity, which is a way to track the length.

The following functions are implemented inside the DoublyLinkedList:

shift(data): This function inserts a new node at the beginning of the linked list. The new node is made the new head, and its next attribute points to the previous head. Since this is a doubly linked list, the previous head's prev attribute is also changed to point to the new node. The capacity is then increased by one.

append(data): This function inserts a new node at the end of the linked list. If the linked list is empty, the new node becomes the head. Otherwise, the function traverses the list until it reaches the last node and attaches the new node to it. The new node's prev attribute is set to point to the previous last node. The capacity is increased by one.

insert(data, n): This function inserts a new node at a specific index in the linked list. First, it checks if the index is within the valid range and is not negative. If the index is 0, the shift function is used, and if the index is equal to the capacity, the append function is used. Otherwise, the function traverses the list until it reaches the desired position and changes both the next and prev pointers so that the new node is inserted between the two existing nodes. The capacity is increased by one.

remove_start(): This function removes the first node from the linked list. It does this by moving the head to the second node in the list and decreasing the capacity by one. Since the new head is now the first node, its prev attribute is changed to None.

remove_end(): This function removes the last node from the linked list. It traverses the list until it reaches the last node. If the node has a previous node, its previous node's next pointer is changed to None. If there is only one node in the list, the head is changed to None. The capacity is then decreased by one.

remove_by_value(n): This function searches for a node containing the given value and removes it from the linked list. It traverses the list until it finds a node with the given value. If the node is the head, the head is moved to the next node and the new head's prev attribute is changed to None. Otherwise, the function changes the previous node's next pointer and the next node's prev pointer so that they skip over the node being removed. The capacity is decreased by one.

str(): This function determines how the linked list should be represented when it is printed. It traverses the list from the head and adds each node's data to a string, with each value separated by a space.

len(): This function allows the built-in len() function to be used on the linked list. It simply returns the capacity.

3. Write a python program to implement a circular linked list which could perform following operations by taking an input from the user, creating a circular linked list, insert in beginning, insert at end, insert at index, delete by value, delete frist node, delete last node, count number of nodes, display or traverse the list, exit.

```
class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None
        self.capacity = 0

    def shift(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            new_node.next = self.head
            return

        curr = self.head
        while curr.next != self.head:
            curr = curr.next

        new_node.next = self.head
        curr.next = new_node
        self.head = new_node

        self.capacity += 1

    def append(self, data):
```

```
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            new_node.next = self.head
            self.capacity += 1
            return

        curr = self.head
        while curr.next != self.head:
            curr = curr.next
        curr.next = new_node
        new_node.next = self.head

        self.capacity += 1

    def insert(self, data, n):
        if n > self.capacity:
            print("Error: index out of bounds")
            return
        if n < 0:
            print("Negative Index")
            return

        if n == 0:
            self.shift(data)
            return

        if n == self.capacity:
            self.append(data)
            return

        new_node = Node(data)
        curr = self.head
        pos = 0
        while pos < n - 1:
            curr = curr.next
            pos += 1

        new_node.next = curr.next
        curr.next = new_node
        self.capacity += 1

    def remove_start(self):
        if not self.head:
            return

        if self.head.next == self.head:
            self.head = None
            self.capacity -= 1
```

```
        return

    curr = self.head

    while curr.next != self.head:
        curr = curr.next

    self.head = self.head.next
    curr.next = self.head

    self.capacity -= 1

def remove_end(self):
    if not self.head:
        return

    if self.head.next == self.head:
        self.head = None
        self.capacity -= 1
        return

    curr = self.head

    while curr.next.next != self.head:
        curr = curr.next

    curr.next = self.head

    self.capacity -= 1


def remove_by_value(self, n):
    if not self.head:
        return

    if self.head.data == n:
        self.remove_start()
        return

    curr = self.head

    while curr.next != self.head and curr.next.data != n:
        curr = curr.next

    if curr.next == self.head:
        print("not found")
        return
```

```
            curr.next = curr.next.next

            self.capacity -= 1



    def __str__(self):
        if not self.head:
            return ''
        current = self.head
        str = ''
        while True:
            str += f'{current.data} '
            current = current.next
            if current == self.head:
                break
        return str.rstrip()


    def __len__(self):
        return self.capacity

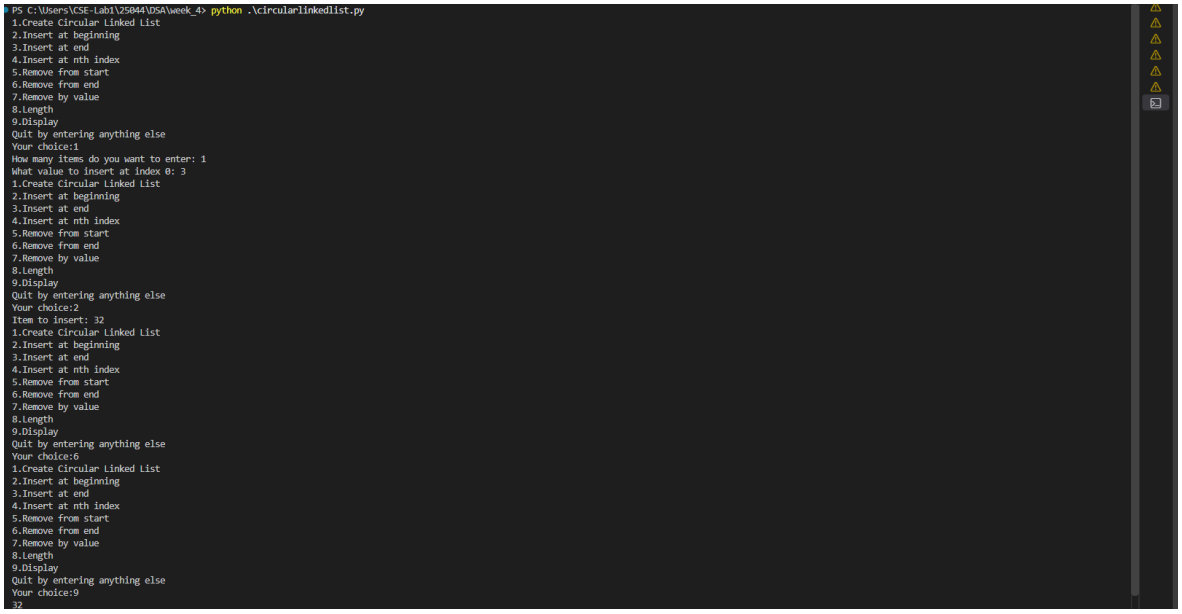

cll = None

menu="'1.Create Circular Linked List\n2.Insert at beginning\n3.Insert at end\n4.Insert at nth index\n5.Remove from start\n6.Remove from end\n7.Remove by value\n8.Length\n9.Display\nQuit by entering anything else'"

while True:
    print(menu)
    s = input("Your choice:")
    if s == '1':
        cll = CircularLinkedList()
        n = int(input("How many items do you want to enter: "))
        for i in range(n):
            x = eval(input(f'What value to insert at index {i}: '))
            cll.append(x)
    elif s == '2':
        x = eval(input("Item to insert: "))
        cll.shift(x)
    elif s == '3':
        x = eval(input("Item to insert: "))
        cll.append(x)
    elif s == '4':
        x = eval(input("Item to insert: "))
        y = int(input(f'Index to insert at (0 to {cll.capacity}): '))
        cll.insert(x, y)
```

```
    elif s == '5':
        cll.remove_start()
    elif s == '6':
        cll.remove_end()
    elif s == '7':
        x = eval(input("what value do you want removed: "))
        cll.remove_by_value(x)
    elif s == '8':
        print(len(cll))
    elif s == '9':
        print(cll)
    else:
        break
```

Output:



Explanation: The following code demonstrates a Circular Linked List in python. There are two classes which are created: Node and CircularLinkedList. Node is the most basic unit that exists inside a linked list and contains two attributes, the data it holds and the next node it points to. The CircularLinkedList's only attributes are the head node and the capacity, which is a way to track the length. Unlike a normal singly linked list, the last node in a circular linked list points back to the head instead of pointing to None.

The following functions are implemented inside the CircularLinkedList:

shift(data): This function inserts a new node at the beginning of the linked list. If the linked list is empty, the new node becomes the head and its next attribute points back to itself. Otherwise, the function traverses the list until it reaches the last node, then makes the new node point to the previous head and changes the last node's next pointer to the new node. The new node is then made the new head. The capacity is increased by one.

append(data): This function inserts a new node at the end of the linked list. If the linked list is empty, the new node becomes the head and points back to itself. Otherwise, the function traverses the list until it reaches the last node and attaches the new node to it. The new node's next attribute is then set to the head so that the circular structure is maintained. The capacity is increased by one.

insert(data, n): This function inserts a new node at a specific index in the linked list. First, it checks if the index is within the valid range and is not negative. If the index is 0, the shift function is used, and if the index is equal to the capacity, the append function is used. Otherwise, the function traverses the list until it reaches the node before the desired position and changes its next pointer so that the new node is inserted. The new node then points to the node that was previously at that position. The capacity is increased by one.

remove_start(): This function removes the first node from the linked list. If there is only one node, the head is changed to None. Otherwise, the function traverses the list until it reaches the last node, moves the head to the second node, and changes the last node's next pointer to the new head. This keeps the circular structure of the linked list intact. The capacity is decreased by one.

remove_end(): This function removes the last node from the linked list. If there is only one node, the head is changed to None. Otherwise, the function traverses the list until it reaches the second-to-last node. Its next pointer is then changed to the head, removing the last node while maintaining the circular structure. The capacity is decreased by one.

remove_by_value(n): This function searches for a node containing the given value and removes it from the linked list. If the value is found in the head node, the remove_start function is used. Otherwise, the function traverses the list while checking the next node for the given value. When the value is found, the current node's next pointer is changed to skip over the node being removed. If the value is not found, it prints "not found". The capacity is decreased by one.

str(): This function determines how the linked list should be represented when it is printed. It traverses the list starting from the head and adds each node's data to a string, with each value separated by a space. Since the list is circular, the function continues until it reaches the head again instead of checking for None.

len(): This function allows the built-in len() function to be used on the linked list. It simply returns the capacity.

Week 5

Write a python program to implement a stack using an array, linked list. Also implemented the stack operations such as push, pop, peek and display the stack elements.

1. Using stack

```
class Stack:
    def __init__(self, capacity=100):
        self.stack = []
        self.capacity = capacity

    def new(self):
        self.stack = []
        n = int(input("how long should the stack be: "))
        self.capacity = n
        l = [int(input(f"entry {i+1}: ")) for i in range(int(input("how many entries do
you want: ")))]
        for i in l:
            self.push(i)

    def push(self, item):
        if len(self.stack) >= self.capacity:
            print("stack overflow")
            return
        self.stack.append(item)

    def pop(self):
        if len(self.stack) == 0:
            print("stack underflow")
            return
        self.stack.pop()

    def peek(self):
        if len(self.stack) == 0:
            print("empty stack")
            return

        return self.stack[-1]

    def __str__(self):
        return str(self.stack)

s = Stack()
while True:
    c = input("1. New List.\n2. Push item into stack. \n3. Pop items.\n4. Peek the first
entry.\n5. Print the stack.\nEnter your choice: ")
    if c == '1':
        s.new()
    elif c == '2':
```

```
        n = int(input('item to enter: '))
        s.push(n)
    elif c == '3':
        s.pop()
    elif c == '4':
        print(s.peek())
    elif c == '5':
        print(s)
    else:
        break
```

Output:

```
PS C:\Users\CSE-Lab1\25044\DSA\week_5> python3 .\stack_array.py
1. New List.
2. Push item into stack.
3. Pop items.
4. Peek the first entry.
5. Print the stack.
Enter your choice: 1
how long should the stack be: 20
how many entries do you want: 5
entry 1: 32
entry 2: 867
entry 3: 386
entry 4: 54
entry 5: 83
1. New List.
2. Push item into stack.
3. Pop items.
4. Peek the first entry.
5. Print the stack.
Enter your choice: 5
[32, 867, 386, 54, 83]
1. New List.
2. Push item into stack.
3. Pop items.
4. Peek the first entry.
5. Print the stack.
Enter your choice: 4
83
1. New List.
2. Push item into stack.
3. Pop items.
4. Peek the first entry.
5. Print the stack.
Enter your choice: 2
item to enter: 32
1. New List.
2. Push item into stack.
3. Pop items.
4. Peek the first entry.
5. Print the stack.
Enter your choice: 4
32
1. New List.
2. Push item into stack.
3. Pop items.
4. Peek the first entry.
5. Print the stack.
Enter your choice: 3
1. New List.
2. Push item into stack.
3. Pop items.
4. Peek the first entry.
5. Print the stack.
Enter your choice: 5
[32, 867, 386, 54, 83]
1. New List.
2. Push item into stack.
3. Pop items.
4. Peek the first entry.
5. Print the stack.
Enter your choice:
```

Explanation: The following code demonstrates a fixed size stack using array in python. A Stack uses the LIFO (last in first out principal) to store it accesses its object, which means the most recent item to be inserted will also be the first one to get out, much like stacking books on top of each other. The base class keeps track of two things, the maximum capacity and the array itself.

new(): this function is used to create a new stack and lets the user to input the maximum capacity and how many elements to start out with.

push(): This function first checks if the current length of the stack is more than the maximum capacity, if it is more than the max, it throws an error called “Stack Overflow” and returns back to the menu, without adding the element into the stack.

pop(): This function first checks if the current length of the stack is 0 or less, basically to check if the list is empty, and if that is true, it throws an error called “Stack Underflow” and returns back to the menu.

peek(): This function is used to fetch the element at top most of the stack, which translates to the element at the last index in the array which it is stored in.

str(): This function determines how the stack should be printed, and for now, all it does is print the array itself.

2. Using Linked List

```
class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.head = None
        self.size = 0

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
        self.size += 1

    def pop(self):
        if not self.head:
            return
        self.head = self.head.next
        self.size -= 1


    def __str__(self):
        current = self.head
        str = ''
        while current:
            str += f'{current.data} '
            current = current.next
        return str.rstrip()

    def peek(self):
        return self.head.data
```

```
    def __len__(self):
        return self.size

s = Stack()
while True:
    c = input("1. New List.\n2. Push item into stack. \n3. Pop items.\n4. Peek the first
entry.\n5. Print the stack.\nEnter your choice: ")
    if c == '1':
        n = int(input("How many items do you want to enter: "))
        for i in range(n):
            x = int(input(f"What value to insert at index {i}: "))
            s.push(x)
    elif c == '2':
        n = int(input('item to enter: '))
        s.push(n)
    elif c == '3':
        s.pop()
    elif c == '4':
        print(s.peek())
    elif c == '5':
        print(s)
    else:
        break
```

Output

```
PS C:\Users\CSE-Lab1\25044\DSA\week_5> python .\stack_ll.py
 1. New List.
 2. Push item into stack.
 3. Pop items.
 4. Peek the first entry.
 5. Print the stack.
 Enter your choice: 1
 How many items do you want to enter: 5
 What value to insert at index 0: 324
 What value to insert at index 1: 453
 What value to insert at index 2: 34
 What value to insert at index 3: 456
 What value to insert at index 4: 4
 1. New List.
 2. Push item into stack.
 3. Pop items.
 4. Peek the first entry.
 5. Print the stack.
 Enter your choice: 4
 4
 1. New List.
 2. Push item into stack.
 3. Pop items.
 4. Peek the first entry.
 5. Print the stack.
 Enter your choice: 5
 4 456 34 453 324
 1. New List.
 2. Push item into stack.
 3. Pop items.
 4. Peek the first entry.
 5. Print the stack.
 Enter your choice: 2
 item to enter: 32
 1. New List.
 2. Push item into stack.
 3. Pop items.
 4. Peek the first entry.
 5. Print the stack.
 Enter your choice: 3
 1. New List.
 2. Push item into stack.
 3. Pop items.
 4. Peek the first entry.
 5. Print the stack.
 Enter your choice: 5
 4 456 34 453 324
 1. New List.
 2. Push item into stack.
 3. Pop items.
 4. Peek the first entry.
 5. Print the stack.
 Enter your choice: e
PS C:\Users\CSE-Lab1\25044\DSA\week_5>
```

Explanation: The following code demonstrates a dynamic stack using linked list in python. A Stack uses the LIFO (last in first out principal) to store it accesses its object, which means the most recent item to be inserted will also be the first one to get out, much like stacking books on top of each other. The base class keeps track of two things, the head of stack, and the length, to keep accessing length O(1).

push(): This function inserts a new node at the beginning of the stack. The new node is made the new head, and its next attx`ribute points to the previous head. The size is then increased by one.

pop(): This function removes the top most element of the stack. It does this by moving the head to the second node in the list. If the list is empty, nothing happens. The size then decreased by one.

peak(): This function retrieves the values of the head of the stack.

__str__(): This function determines how the linked list should be represented when it is printed. It traverses the list from the head and adds each node's data to a string, with each value separated by a space.

__len__(): This function allows the built-in len() function to be used on the linked list. It simply returns the capacity.

Week 6

1. Write a program to implement queue and its operations such as enqueue, dequeue, peek, display front and rear approach. Implement using array and linked list data structures.

A. Using arrays

```
class Queue:
    def __init__(self, capacity):
        self.capacity = capacity
        self.front = -1
        self.rear = -1
        self.queue = [None] * capacity

    def enqueue(self, item):
        if self.rear >= self.capacity - 1:
            print("queue overflow")
            return
        if self.front == -1 and self.rear == -1:
            self.front += 1
        self.rear += 1
        self.queue[self.rear] = item

    def dequeue(self):
        if self.front > self.rear:
            print("queue underflow")
            self.front = -1
            self.rear = -1
            return
        self.queue[self.front] = None
        self.front += 1

    def peek(self):
        return self.queue[self.front]

    def display(self):
        print(self.queue[self.front:self.rear + 1])

q = Queue(20)

while True:
    x = input('1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: ')
    if x == '1':
        q.enqueue(int(input("Element to enter: ")))
    elif x == '2':
        q.dequeue()
```

```
    elif x == '3':
        print(q.peek())
    elif x == '4':
        q.display()
    else:
        break
```

```
PS C:\Users\CSE-Lab1\25044\DSA\week_6> python3 .\q_arr.py
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 1
Element to enter: 43
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 1
Element to enter: 34
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 4
[43, 34]
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 2
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 4
[34]
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 5
PS C:\Users\CSE-Lab1\25044\DSA\week_6>
```

Explanation:

Explanation: The following code demonstrates a queue using an array in Python. A Queue uses the FIFO (first in first out) principle to store and access its objects, which means the first item to be inserted will also be the first item to be removed, much like people standing in a line. The `Queue` class keeps track of three main things: the capacity of the queue, the front position, and the rear position. The `front` keeps track of the first element in the queue, while the `rear` keeps track of the most recently inserted element.

`enqueue()`: This function adds a new item to the end of the queue. First, it checks if the queue has reached its maximum capacity. If it has, it prints "queue overflow" and stops the operation. If the queue is empty, the `front` is moved from `-1` to `0`. The `rear` is then increased by one, and the new item is stored at that position in the queue.

`dequeue()`: This function removes the first item from the queue. It checks if the `front` has moved past the `rear`, which means there are no more items in the queue. If the queue is empty, it prints "queue underflow" and resets both `front` and `rear` to `-1`. Otherwise, the item at the front of the queue is replaced with `None`, and the `front` is moved forward by one position.

`peek()`: This function retrieves the value at the front of the queue without removing it. It accesses the element stored at the current `front` position.

`display()`: This function displays all the current elements in the queue. It uses list slicing from the `front` position to the `rear` position and prints the resulting list.

`__init__()`: This function initializes the queue. It takes the maximum capacity as an argument and creates a list of that size, initially filled with `None`. The `front` and `rear` are both set to `-1` to indicate that the queue is initially empty.

The code at the bottom creates a queue with a capacity of 20 and repeatedly asks the user whether they want to enqueue an item, dequeue an item, peek at the front item, display the queue, or exit the program.

B. Using linked lists

```
class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class Queue:
    def __init__(self):
        self.head = None
        self.capacity = 0

    def enqueue(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            self.capacity += 1
            return

        curr = self.head
        while curr.next:
            curr = curr.next
        curr.next = new_node
        self.capacity += 1

    def dequeue(self):
        if not self.head:
            return
        self.head = self.head.next
        self.capacity -= 1

    def peek(self):
        if not self.head:
            return
        return self.head.data

    def display(self):
        current = self.head
        str = ''
        while current:
            str += f'{current.data} '
            current = current.next
        print("[" +  str.rstrip() + "]")
```

```
q = Queue()

while True:
    x = input('1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: ')
    if x == '1':
        q.enqueue(int(input("Element to enter: ")))
    elif x == '2':
        q.dequeue()
    elif x == '3':
        print(q.peek())
    elif x == '4':
        q.display()
    else:
        break
```

```
PS C:\Users\CSE-Lab1\25044\DSA\week_6> python3 .\q_ll.py
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 1
Element to enter: 25
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 1
Element to enter: 456
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 1
Element to enter: 324
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 2
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 4
[456 324]
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 5
PS C:\Users\CSE-Lab1\25044\DSA\week_6>
```

Explanation: The following code demonstrates a dynamic queue using a linked list in Python. A Queue uses the FIFO (first in first out) principle to store and access its objects, which means the first item to be inserted will also be the first item to be removed, much like people standing in a line. The Queue class keeps track of two things: the head of the queue and the capacity, which keeps track of the number of items currently stored in the queue.

enqueue(): This function inserts a new node at the end of the queue. First, a new node is created with the given data. If the queue is empty, the new node becomes the head. Otherwise, the function traverses the linked list until it reaches the last node, and the new node is attached to its next attribute. The capacity is then increased by one.

dequeue(): This function removes the first item from the queue. If the queue is empty, nothing happens. Otherwise, the head is moved to the next node in the linked list, effectively removing the previous first node. The capacity is then decreased by one.

peek(): This function retrieves the value of the first item in the queue without removing it. If the queue is empty, nothing is returned. Otherwise, it returns the data stored in the node at the head.

display(): This function displays all the current elements in the queue. It starts at the head and traverses through each node using the next attribute. Each node's data is added to a string, with the values separated by spaces, and the resulting string is printed inside square brackets.

__init__(): This function initializes the queue. The head is initially set to None because the queue starts empty, and capacity is set to 0 to keep track of the number of elements currently in the queue.

Node: This class represents each individual element in the linked list. Each node stores two things: its data and a next attribute that points to the next node in the queue. Initially, next is set to None because the node does not point to another node when it is created.

The code at the bottom creates an empty queue and repeatedly asks the user whether they want to enqueue an item, dequeue an item, peek at the first item, display the queue, or exit the program.

2. Write a program to implement circular queue and its operations such as enqueue, dequeue, peek, display front and rear approach. Implement using array and linked list data structures.

A. Using arrays

```
class CircularQueue:
    def __init__(self, capacity):
        self.capacity = capacity
        self.front = -1
        self.rear = -1
        self.queue = [None] * capacity

    def enqueue(self, item):
        if (self.rear + 1) % self.capacity == self.front:
            print("queue overflow")
            return
        if self.front == -1 and self.rear == -1:
            self.front += 1
        self.rear = (self.rear + 1) % self.capacity
        self.queue[self.rear] = item

    def dequeue(self):
        if self.front == -1:
            print("queue underflow")
            return

        self.queue[self.front] = None

        if self.front == self.rear:
            self.front = -1
            self.rear = -1
        else:
            self.front = (self.front + 1) % self.capacity



    def peek(self):
```

```
        return self.queue[self.front]

    def display(self):
        if self.rear >= self.front:
            print(self.queue[self.front:self.rear + 1])
        else:
            print(self.queue[self.front:self.capacity] + self.queue[0: self.rear + 1])

q = CircularQueue(20)

while True:
    x = input('1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: ')
    if x == '1':
        q.enqueue(int(input("Element to enter: ")))
    elif x == '2':
        q.dequeue()
    elif x == '3':
        print(q.peek())
    elif x == '4':
        q.display()
    else:
        break
```

```
PS C:\Users\CSE-Lab1\25044\DSA\week_6> python3 .\cq_arr.py
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 1
Element to enter: 423
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 1
Element to enter: 23
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 1
Element to enter: 23
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 1
Element to enter: 34
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 2
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 2
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 4
[23, 34]
1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: 6
PS C:\Users\CSE-Lab1\25044\DSA\week_6>
```

Explanation: The following code demonstrates a circular queue using an array in Python. A Circular Queue uses the FIFO (first in first out) principle to store and access its objects, which means the first item to be inserted will also be the first item to be removed, much like people standing in a line. Unlike a normal queue, a circular queue allows the rear of the queue to wrap around to the beginning of the array when there is available space. The `CircularQueue` class keeps track of three main things: the capacity of the queue, the `front` position, and the `rear` position. The `front` keeps track of the first element, while the `rear` keeps track of the most recently inserted element.

`enqueue()`: This function adds a new item to the rear of the queue. First, it checks whether the next position of the `rear` would be the current `front`. If so, the queue is full and "queue overflow" is printed. If the queue is empty, the `front` is moved from `-1` to `0`. The `rear` is then moved forward using the modulo operator `%`, which allows it to wrap back to the beginning of the array when it reaches the end. The new item is then stored at the updated `rear` position.

`dequeue()`: This function removes the first item from the queue. If the `front` is `-1`, the queue is empty, so "queue underflow" is printed and nothing happens. Otherwise, the element at the `front` position is replaced with `None`. If the `front` and `rear` are at the same position, it means there was only one item in the queue, so both are reset to `-1`. Otherwise, the `front` is moved to the next position using the modulo operator, allowing it to wrap around to the beginning of the array when necessary.

`peek()`: This function retrieves the value at the front of the queue without removing it. It accesses the element stored at the current `front` position.

`display()`: This function displays all the current elements in the circular queue. If the `rear` is greater than or equal to the `front`, the elements can be displayed using a single slice of the array. If the `rear` has wrapped around and is smaller than the `front`, the queue is split into two parts. The first part is taken from `front` to the end of the array, and the second part is taken from the beginning of the array to `rear`. These two parts are then combined and printed.

`__init__()`: This function initializes the circular queue. It takes the maximum capacity as an argument and creates a list of that size, initially filled with `None`. The `front` and `rear` are both set to `-1` to indicate that the queue is initially empty.

The code at the bottom creates a circular queue with a capacity of 20 and repeatedly asks the user whether they want to enqueue an item, dequeue an item, peek at the front item, display the queue, or exit the program. The use of the modulo operator allows the queue to reuse empty positions at the beginning of the array after the `rear` reaches the end.

B. Using linked lists:

```
class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class CircularQueue:
    def __init__(self):
        self.head = None
        self.capacity = 0

    def enqueue(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            new_node.next = self.head
            self.capacity += 1
            return

        curr = self.head
        while curr.next != self.head:
            curr = curr.next
```

```
        curr.next = new_node
        new_node.next = self.head

        self.capacity += 1

    def dequeue(self):
        if not self.head:
            return

        removed_data = self.head.data

        if self.head.next == self.head:
            self.head = None

        else:
            curr = self.head
            while curr.next != self.head:
                curr = curr.next
            self.head = self.head.next
            curr.next = self.head

        self.capacity -= 1
        return removed_data

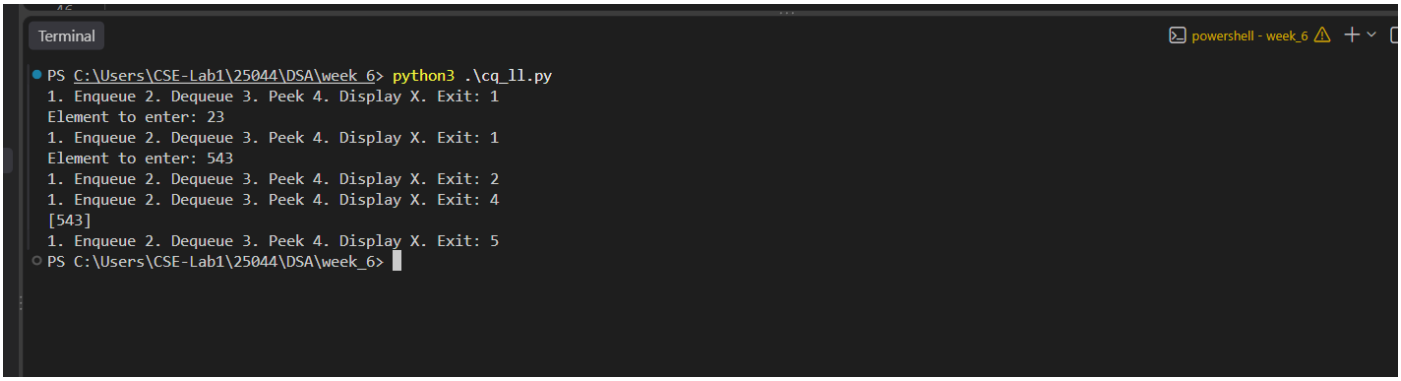

    def peek(self):
        if not self.head:
            return
        return self.head.data

    def display(self):
        if not self.head:
            return ''
        current = self.head
        str = ''
        while True:
            str += f'{current.data} '
            current = current.next
            if current == self.head:
                break
        print("[" +  str.rstrip() + "]")

q = CircularQueue()

while True:
    x = input('1. Enqueue 2. Dequeue 3. Peek 4. Display X. Exit: ')
    if x == '1':
```

```
        q.enqueue(int(input("Element to enter: ")))
    elif x == '2':
        q.dequeue()
    elif x == '3':
        print(q.peek())
    elif x == '4':
        q.display()
    else:
        break
```



Explanation: The following code demonstrates a circular queue using a linked list in Python. A Circular Queue uses the FIFO (first in first out) principle to store and access its objects, which means the first item to be inserted will also be the first item to be removed. Unlike a normal linked-list queue, the last node in a circular queue points back to the first node instead of pointing to `None`. The `CircularQueue` class keeps track of two things: the `head` of the queue and the `capacity`, which keeps track of the number of elements currently stored in the queue.

`enqueue()`: This function inserts a new node at the end of the circular queue. First, a new node is created with the given data. If the queue is empty, the new node becomes the `head`, and its `next` attribute points back to itself, creating the circular connection. If the queue is not empty, the function traverses the linked list until it reaches the last node, which is the node whose `next` points back to the `head`. The new node is then attached to the last node, and its `next` attribute is set to the `head`. The `capacity` is then increased by one.

`dequeue()`: This function removes the first item from the circular queue. If the queue is empty, nothing happens. The data of the current `head` is first stored so it can be returned after removal. If the queue contains only one node, the `head` is set to `None`. Otherwise, the function traverses the list until it reaches the last node. The `head` is then moved to the next node, and the last node's `next` attribute is updated to point to the new `head`, maintaining the circular structure. The `capacity` is then decreased by one, and the removed data is returned.

`peek()`: This function retrieves the value of the first item in the queue without removing it. If the queue is empty, nothing is returned. Otherwise, it returns the `data` stored in the node at the `head`.

`display()`: This function displays all the current elements in the circular queue. Since the last node points back to the `head`, a normal `while current` loop cannot be used because there is no `None` at the end of the

list. Instead, the function uses a `while True` loop and stops when `current` reaches the `head` again. Each node's data is added to a string and the resulting values are printed inside square brackets.

`__init__()`: This function initializes the circular queue. The `head` is initially set to `None` because the queue starts empty, and `capacity` is set to `0` to keep track of the number of elements currently in the queue.

`Node`: This class represents each individual element in the linked list. Each node stores two things: its `data` and a `next` attribute that points to the next node in the queue. In a circular queue, the `next` attribute of the last node points back to the `head` instead of being set to `None`.

The code at the bottom creates an empty circular queue and repeatedly asks the user whether they want to enqueue an item, dequeue an item, peek at the first item, display the queue, or exit the program. The circular structure is maintained by making the last node point back to the first node, allowing the queue to continuously cycle through its elements.

Week 7

1. Write a python program to implement binary tree using an array and a linked list with all of its operations.

A. Using an array

```
class Node:
def __init__(self, value):
self.value = value
self.left = None
self.right = None



class BinaryTree:
def __init__(self):
self.root = None

def find_node(self, value):
if self.root is None:
return None

queue = [self.root]

while queue:
current = queue.pop(0)

if current.value == value:
return current

if current.left is not None:
queue.append(current.left)

if current.right is not None:
queue.append(current.right)

return None

def insert_left(self, value, parentValue):
parent = self.find_node(parentValue)

if parent is None:
print("invalid parent")
return

if parent.left is not None:
print("left child already exists")
return

parent.left = Node(value)
```

```
def insert_right(self, value, parentValue):
parent = self.find_node(parentValue)

if parent is None:
print("invalid parent")
return

if parent.right is not None:
print("right child already exists")
return

parent.right = Node(value)

def remove(self, value):
if self.root is None:
print("tree is empty")
return

if self.root.value == value:
self.root = None
return

queue = [(self.root, None)]

while queue:
current, parent = queue.pop(0)

if current.value == value:
if parent.left == current:
parent.left = None
elif parent.right == current:
parent.right = None

return

if current.left is not None:
queue.append((current.left, current))

if current.right is not None:
queue.append((current.right, current))

print("value not found")

def traverse_inorder(self, node=None):
if node is None:
node = self.root

if node is not None:
self.traverse_inorder(node.left)
print(node.value, end=" ")
self.traverse_inorder(node.right)

def traverse_preorder(self, node=None):
if node is None:
node = self.root
```

```
if node is not None:
print(node.value, end=" ")
self.traverse_preorder(node.left)
self.traverse_preorder(node.right)

def traverse_postorder(self, node=None):
if node is None:
node = self.root

if node is not None:
self.traverse_postorder(node.left)
self.traverse_postorder(node.right)
print(node.value, end=" ")



b = BinaryTree()

while True:
x = input("1. Insert Left 2. Insert Right 3. Remove 4. Inorder 5. Preorder 6. Postorder\nYour choice: ")

if x == '1':
b.insert_left(int(input("Value to Insert: ")), int(input("Parent Value: ")))
elif x == '2':
b.insert_right(int(input("Value to Insert: ")), int(input("Parent Value: ")))
elif x == '3':
b.remove(int(input("Value to remove: ")))
elif x == '4':
b.traverse_inorder()
print()
elif x == '5':
b.traverse_preorder()
print()
elif x == '6':
b.traverse_postorder()
print()
else:
break
```

Output:

```
❯ python3 week_7/bintreearr.py
1.In Order Insert 2. Insert Left 3. Insert Right 4. Remove 5. Inorder 6. Preorder 7. Postorder
Your choice: 1
Value to Insert: 12
1.In Order Insert 2. Insert Left 3. Insert Right 4. Remove 5. Inorder 6. Preorder 7. Postorder
Your choice: 2
Value to Insert: 4
Parent Index: 0
1.In Order Insert 2. Insert Left 3. Insert Right 4. Remove 5. Inorder 6. Preorder 7. Postorder
Your choice: 5
4 12
1.In Order Insert 2. Insert Left 3. Insert Right 4. Remove 5. Inorder 6. Preorder 7. Postorder
Your choice: 3
Value to Insert: 23
Parent Index: 1
1.In Order Insert 2. Insert Left 3. Insert Right 4. Remove 5. Inorder 6. Preorder 7. Postorder
Your choice: 7
23 4 12
1.In Order Insert 2. Insert Left 3. Insert Right 4. Remove 5. Inorder 6. Preorder 7. Postorder
Your choice: d
```

Explanation:

The following code demonstrates the implementation of a binary tree using a Python list. A binary tree is a hierarchical data structure in which each node can have at most two children, called the left child and the right child. In this implementation, the tree is represented using an array-like list instead of linked nodes. The BinaryTree class maintains a list named tree, which initially contains 20 elements, all set to None. Each node is stored at a specific index, where the left child of a node at index i is located at 2 * i + 1, and the right child is located at 2 * i + 2.

init(): This function initializes the binary tree by creating a list of 20 elements, all set to None. These empty positions can be used to store the values of the tree nodes. The list can also be extended when additional space is required for inserting child nodes.

insert(): This function inserts a value into the tree by searching for an available position in the list. It starts from the last index and moves backwards until it finds a position that already contains a value or reaches the beginning of the list. The value is then stored at the identified index. This function does not perform a conventional binary search tree insertion; instead, it uses the existing list contents to determine where the value is placed.

insert_left(): This function inserts a value as the left child of a specified parent node. It first checks whether the parent index is within the bounds of the list and whether the position contains a valid node. If the parent is invalid, an error message is displayed. Otherwise, the left child's index is calculated using 2 * parentIndex + 1. If the calculated index exceeds the current list size, the list is extended with additional None elements. The value is then stored at the calculated index.

insert_right(): This function is intended to insert a value as the right child of a specified parent node. It checks whether the parent index is valid and whether a node exists at that position. If the parent is invalid, an error message is displayed. Otherwise, the function calculates the position for the right child, extends the list if necessary, and stores the value at that position. The correct formula for the right child index is 2 * parentIndex + 2.

remove_index(): This function removes a node from the tree by setting the value at the specified index to None. This makes the position appear empty without changing the size of the list. The function does not automatically remove or rearrange the node's children.

traverse_inorder(): This function performs an inorder traversal of the binary tree. It first checks whether the current index is within the list bounds and contains a valid node. If the condition is satisfied, it recursively traverses the left subtree, prints the current node's value, and then traverses the right subtree. Therefore, the traversal follows the order **Left → Root → Right**.

traverse_preorder(): This function performs a preorder traversal of the binary tree. It first prints the current node's value, then recursively traverses the left subtree, followed by the right subtree. The traversal follows the order **Root → Left → Right**. The function stops exploring a branch when it reaches an index outside the list or a position containing None.

traverse_postorder(): This function performs a postorder traversal of the binary tree. It recursively traverses the left subtree first, then the right subtree, and finally prints the current node's value. The traversal follows the order **Left → Right → Root**. As with the other traversal functions, recursion continues only when the current index is valid and contains a node.

The code at the bottom creates an object of the BinaryTree class and repeatedly displays a menu that allows the user to insert a value, insert a left or right child, remove a node at a specified index, or perform one of the three tree traversals. The program accepts the user's choice and calls the corresponding function. The loop continues until the user enters an option that is not listed in the menu, at which point the program terminates.

The implementation demonstrates how a binary tree can be represented using

B. Using Linked Lists

```
class Node:
def __init__(self, value):
self.value = value
self.left = None
self.right = None



class BinaryTree:
def __init__(self):
self.root = None

def insert_root(self, value):
if self.root is not None:
print("root already exists")
return

self.root = Node(value)

def find_node(self, value):
```

```
if self.root is None:
return None

queue = [self.root]

while queue:
current = queue.pop(0)

if current.value == value:
return current

if current.left is not None:
queue.append(current.left)

if current.right is not None:
queue.append(current.right)

return None

def insert_left(self, value, parentValue):
parent = self.find_node(parentValue)

if parent is None:
print("invalid parent")
return

if parent.left is not None:
print("left child already exists")
return

parent.left = Node(value)

def insert_right(self, value, parentValue):
parent = self.find_node(parentValue)

if parent is None:
print("invalid parent")
return

if parent.right is not None:
print("right child already exists")
return

parent.right = Node(value)

def remove(self, value):
if self.root is None:
print("tree is empty")
return

if self.root.value == value:
self.root = None
return
```

```
queue = [(self.root, None)]

while queue:
current, parent = queue.pop(0)

if current.value == value:
if parent.left == current:
parent.left = None
elif parent.right == current:
parent.right = None
return

if current.left is not None:
queue.append((current.left, current))

if current.right is not None:
queue.append((current.right, current))

print("value not found")

def traverse_inorder(self, node=None):
if node is None:
node = self.root
if node is None:
return

if node.left is not None:
self.traverse_inorder(node.left)

print(node.value, end=" ")

if node.right is not None:
self.traverse_inorder(node.right)

def traverse_preorder(self, node=None):
if node is None:
node = self.root
if node is None:
return

print(node.value, end=" ")

if node.left is not None:
self.traverse_preorder(node.left)

if node.right is not None:
self.traverse_preorder(node.right)

def traverse_postorder(self, node=None):
if node is None:
node = self.root
if node is None:
return

if node.left is not None:
```

```
self.traverse_postorder(node.left)

if node.right is not None:
self.traverse_postorder(node.right)

print(node.value, end=" ")



b = BinaryTree()

while True:
x = input("1. Insert Root 2. Insert Left 3. Insert Right 4. Remove 5. Inorder 6. Preorder 7. Postorder 8. Exit\nYour choice: ")

if x == '1':
b.insert_root(int(input("Root value: ")))
elif x == '2':
b.insert_left(int(input("Value to Insert: ")), int(input("Parent Value: ")))
elif x == '3':
b.insert_right(int(input("Value to Insert: ")), int(input("Parent Value: ")))
elif x == '4':
b.remove(int(input("Value to remove: ")))
elif x == '5':
b.traverse_inorder()
print()
elif x == '6':
b.traverse_preorder()
print()
elif x == '7':
b.traverse_postorder()
print()
elif x == '8':
break
else:
print("Invalid choice")
```

Output:

```
23cse113/dsa on  master [!?]
❯ python3 week_7/bintreell.py
1. Insert Root 2. Insert Left 3. Insert Right 4. Remove 5. Inorder 6. Preorder 7. Postorder 8. Exit
Your choice: 1
Root value: 232
1. Insert Root 2. Insert Left 3. Insert Right 4. Remove 5. Inorder 6. Preorder 7. Postorder 8. Exit
Your choice: 2
Value to Insert: 232
Parent Value: 232
1. Insert Root 2. Insert Left 3. Insert Right 4. Remove 5. Inorder 6. Preorder 7. Postorder 8. Exit
Your choice: 3
Value to Insert: 221
Parent Value: 232
1. Insert Root 2. Insert Left 3. Insert Right 4. Remove 5. Inorder 6. Preorder 7. Postorder 8. Exit
Your choice: 7
232 221 232
1. Insert Root 2. Insert Left 3. Insert Right 4. Remove 5. Inorder 6. Preorder 7. Postorder 8. Exit
Your choice: 5
232 232 221
1. Insert Root 2. Insert Left 3. Insert Right 4. Remove 5. Inorder 6. Preorder 7. Postorder 8. Exit
Your choice: 11
Invalid choice
1. Insert Root 2. Insert Left 3. Insert Right 4. Remove 5. Inorder 6. Preorder 7. Postorder 8. Exit
Your choice: 8
```

Explanation:

Explanation:

The following code demonstrates the implementation of a binary tree using a linked representation in Python. A binary tree is a hierarchical data structure in which each node can have at most two children, called the left child and the right child. Unlike an array-based binary tree, this implementation uses individual nodes connected through references. The BinaryTree class maintains a root node, which acts as the starting point of the tree. Each node contains a value and references to its left and right children. The program supports inserting nodes, searching for nodes, removing nodes, and performing different types of tree traversals.

init(): This function initializes the binary tree by setting the root attribute to None. This indicates that the tree is initially empty. Once a root node is inserted, the root attribute stores a reference to that node, allowing the other functions to access and traverse the tree.

insert_root(): This function inserts the root node into the binary tree. It first checks whether the root already exists. If the tree already contains a root node, an error message is displayed, and the function returns without making any changes. Otherwise, a new node is created using the given value and assigned as the root of the tree.

find_node(): This function searches for a node containing a specified value in the binary tree. If the tree is empty, it returns None. Otherwise, it creates a queue containing the root node and performs a breadth-first search. The function repeatedly removes the first node from the queue and compares its value with the required value. If the values match, the node is returned. Otherwise, its left and right children, if present, are added to the queue. This process continues until the required node is found or the queue becomes empty. If no matching node exists, the function returns None.

insert_left(): This function inserts a new node as the left child of a specified parent node. It first calls the find_node() function to locate the parent using its value. If the parent does not exist, an error message is displayed. If the parent already has a left child, the function displays a message indicating that the position is occupied. Otherwise, a new node is created with the given value and assigned to the left attribute of the parent node.

insert_right(): This function inserts a new node as the right child of a specified parent node. It first searches for the parent node using the find_node() function. If the parent cannot be found, an error message is displayed. If the parent already has a right child, the function displays a message and returns without inserting the node. Otherwise, a new node is created and assigned to the right attribute of the parent, establishing the connection between the two nodes.

remove(): This function removes a node containing a specified value from the binary tree. It first checks whether the tree is empty and displays an error message if no nodes exist. If the root contains the required value, the root is set to None, removing the entire tree. Otherwise, the function uses a queue to perform a breadth-first search while keeping track of each node and its parent. When the required node is found, the corresponding left or right reference of its parent is set to None. This disconnects the selected node and its

entire subtree from the tree. If the value is not found after searching all nodes, an appropriate message is displayed.

traverse_inorder(): This function performs an inorder traversal of the binary tree using recursion. If no starting node is provided, the traversal begins from the root. The function first recursively visits the left subtree, then prints the current node's value, and finally visits the right subtree. Therefore, the traversal follows the order **Left → Root → Right**. The recursive calls continue until all accessible nodes have been visited.

traverse_preorder(): This function performs a preorder traversal of the binary tree. It begins at the root when no starting node is specified. The function first prints the current node's value, then recursively visits the left subtree, followed by the right subtree. The traversal follows the order **Root → Left → Right**. This method processes each parent node before visiting its children.

traverse_postorder(): This function performs a postorder traversal of the binary tree using recursion. It first recursively visits the left subtree, followed by the right subtree, and finally prints the current node's value. The traversal follows the order **Left → Right → Root**. This ensures that the child nodes are processed before their parent node.

The code at the bottom creates an empty binary tree and repeatedly displays a menu allowing the user to insert a root node, insert left or right children, remove a node, or perform inorder, preorder, and postorder traversals. The program accepts the user's choice and calls the corresponding function. For insertion and removal operations, the user is asked to enter the required node values. The loop continues until the user selects option 8 to exit the program. If an invalid option is entered, an error message is displayed, and the menu is shown again.

The implementation demonstrates how a binary tree can be constructed and managed using dynamically linked nodes instead of a fixed-size list. The find_node() and remove() functions use breadth-first search to locate nodes, while the traversal functions use recursion to visit the tree in different orders. The linked representation allows nodes to be added without allocating a fixed amount of space beforehand.

Week 8

1. wap a program to implement binary search trees with its operations: insert, delete, traversal and searching,

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None


class BinarySearchTree:
```

```
    def __init__(self):
        self.root = None

    def insert(self, root, value):
        if root is None:
            return Node(value)

        if root.value == value:
            return root

        if root.value < value:
            root.right = self.insert(root.right, value)
        else:
            root.left = self.insert(root.left, value)

        return root

    def traverse_inorder(self, node=None):
        if node is None:
            node = self.root

        if node is None:
            return

        self._inorder(node)

    def _inorder(self, node):
        if node is None:
            return

        self._inorder(node.left)
        print(node.value, end=" ")
        self._inorder(node.right)

    def search(self, root, value):
        if root is None or root.value == value:
            return root

        if root.value < value:
            return self.search(root.right, value)

        return self.search(root.left, value)

    def delete(self, root, value):
        if root is None:
            return None
```

```
        if value < root.value:
            root.left = self.delete(root.left, value)

        elif value > root.value:
            root.right = self.delete(root.right, value)

        else:
            if root.left is None:
                return root.right

            if root.right is None:
                return root.left

            curr = root.right
            while curr.left is not None:
                curr = curr.left

            root.value = curr.value
            root.right = self.delete(root.right, curr.value)

        return root


b = BinarySearchTree()

while True:
    x = input("1. Insert | 2. Remove | 3. Traverse | 4. Search | 5. Exit: ")

    if x == '1':
        value = int(input("Enter value to insert: "))
        b.root = b.insert(b.root, value)

    elif x == '2':
        value = int(input("Value to remove: "))
        b.root = b.delete(b.root, value)

    elif x == '3':
        b.traverse_inorder()
        print()

    elif x == '4':
        value = int(input("Value to search for: "))
        print("Found" if b.search(b.root, value)
              else "Not Found")

    elif x == '5':
        break
```

```
    else:
        print("Invalid choice")
```

Output:

```
PS C:\Users\CSE-Lab1\25044\DSA\week_8> python3 .\bst.py
1. Insert | 2. Remove | 3. Traverse | 4. Search | 5. Exit: 1
Enter value to insert: 23
1. Insert | 2. Remove | 3. Traverse | 4. Search | 5. Exit: 1
Enter value to insert: 654
1. Insert | 2. Remove | 3. Traverse | 4. Search | 5. Exit: 1
Enter value to insert: 4543
1. Insert | 2. Remove | 3. Traverse | 4. Search | 5. Exit: 1
Enter value to insert: 1
1. Insert | 2. Remove | 3. Traverse | 4. Search | 5. Exit: 3
1 23 654 4543
1. Insert | 2. Remove | 3. Traverse | 4. Search | 5. Exit: 2
Value to remove: 23
1. Insert | 2. Remove | 3. Traverse | 4. Search | 5. Exit: 2
Value to remove: 1
1. Insert | 2. Remove | 3. Traverse | 4. Search | 5. Exit: 1
Enter value to insert: 1
1. Insert | 2. Remove | 3. Traverse | 4. Search | 5. Exit: 4
Value to search for: 23
Not Found
1. Insert | 2. Remove | 3. Traverse | 4. Search | 5. Exit: 4
Value to search for: 1
Found
1. Insert | 2. Remove | 3. Traverse | 4. Search | 5. Exit: 3
1 654 4543
1. Insert | 2. Remove | 3. Traverse | 4. Search | 5. Exit: 5
PS C:\Users\CSE-Lab1\25044\DSA\week_8> S
```

Explanation: The following code demonstrates a Binary Search Tree (BST) in Python. A Binary Search Tree is a hierarchical data structure in which each node contains a value and references to its left and right child nodes. For every node, values smaller than the node are stored in the left subtree, while values greater than the node are stored in the right subtree. This property makes searching, insertion, and deletion efficient in many cases.

Node class: This class represents a single node in the binary search tree. The __init__() function initializes the node with a value and sets its left and right children to None.

init(): This function initializes the BinarySearchTree class. The root variable is set to None, indicating that the tree is initially empty.

insert(): This function inserts a new value into the binary search tree. If the current node is None, a new node is created. If the value already exists, the function returns the existing node without inserting a duplicate. If the value is greater than the current node's value, it is inserted into the right subtree. Otherwise, it is inserted into the left subtree. The function returns the updated root of the subtree.

traverse_inorder(): This function performs an inorder traversal of the tree. If no node is provided, it starts from the root. If the tree is empty, it returns without doing anything. Otherwise, it calls the _inorder() helper function to traverse the tree.

_inorder(): This recursive helper function visits the left subtree, prints the current node's value, and then visits the right subtree. In a Binary Search Tree, inorder traversal displays the values in ascending order.

search(): This function searches for a given value in the binary search tree. If the current node is None or contains the required value, it returns the node. If the value is greater than the current node's value, the search continues in the right subtree. Otherwise, it continues in the left subtree. If the value is not present, the function returns None.

delete(): This function removes a specified value from the binary search tree. If the node is None, it returns None. If the value is smaller or greater than the current node's value, the function recursively searches the left or right subtree, respectively. When the value is found, there are three cases:

1. No left child: The node is replaced by its right child.
2. No right child: The node is replaced by its left child.
3. Two children: The smallest value in the right subtree is found and copied into the current node. The original node containing that smallest value is then deleted. This preserves the Binary Search Tree property.

Week 9

1. wap a python program to implement avl trees with its operations: insert, delete, traversal

```
class Node:
def __init__(self, value):
self.value = value
self.left = None
self.right = None
self.height = 1



class AVLTree:
def height(self, node):
if node is None:
return 0
return node.height

def balance(self, node):
if node is None:
return 0
return self.height(node.left) - self.height(node.right)

def right_rotate(self, y):
x = y.left
t2 = x.right

x.right = y
y.left = t2

y.height = 1 + max(self.height(y.left), self.height(y.right))
x.height = 1 + max(self.height(x.left), self.height(x.right))

return x

def left_rotate(self, x):
y = x.right
t2 = y.left

y.left = x
x.right = t2

x.height = 1 + max(self.height(x.left), self.height(x.right))
y.height = 1 + max(self.height(y.left), self.height(y.right))

return y

def insert(self, root, value):
if root is None:
return Node(value)

if value < root.value:
```

```
root.left = self.insert(root.left, value)
elif value > root.value:
root.right = self.insert(root.right, value)
else:
return root

root.height = 1 + max(self.height(root.left), self.height(root.right))
b = self.balance(root)

# LL Case
if b > 1 and value < root.left.value:
return self.right_rotate(root)

# RR Case
if b < -1 and value > root.right.value:
return self.left_rotate(root)

# LR Case
if b > 1 and value > root.left.value:
root.left = self.left_rotate(root.left)
return self.right_rotate(root)

# RL Case
if b < -1 and value < root.right.value:
root.right = self.right_rotate(root.right)
return self.left_rotate(root)

return root

def min_value_node(self, node):
current = node
while current.left is not None:
current = current.left
return current

def delete(self, root, value):
if root is None:
return None

if value < root.value:
root.left = self.delete(root.left, value)
elif value > root.value:
root.right = self.delete(root.right, value)
else:
if root.left is None:
return root.right
elif root.right is None:
return root.left

temp = self.min_value_node(root.right)
root.value = temp.value
root.right = self.delete(root.right, temp.value)

root.height = 1 + max(self.height(root.left), self.height(root.right))
b = self.balance(root)
```

```
# LL Case
if b > 1 and self.balance(root.left) >= 0:
return self.right_rotate(root)

# LR Case
if b > 1 and self.balance(root.left) < 0:
root.left = self.left_rotate(root.left)
return self.right_rotate(root)

# RR Case
if b < -1 and self.balance(root.right) <= 0:
return self.left_rotate(root)

# RL Case
if b < -1 and self.balance(root.right) > 0:
root.right = self.right_rotate(root.right)
return self.left_rotate(root)

return root

def search(self, root, value):
if root is None or root.value == value:
return root
if value < root.value:
return self.search(root.left, value)
return self.search(root.right, value)

def inorder(self, root):
if root is not None:
self.inorder(root.left)
print(root.value, end=" ")
self.inorder(root.right)



tree = AVLTree()
root = None

while True:
x = input("1. Insert | 2. Remove | 3. Traverse | 4. Search | 5. Exit: ")

if x == '1':
value = int(input("Enter value to insert: "))
root = tree.insert(root, value)

elif x == '2':
value = int(input("Value to remove: "))
root = tree.delete(root, value)

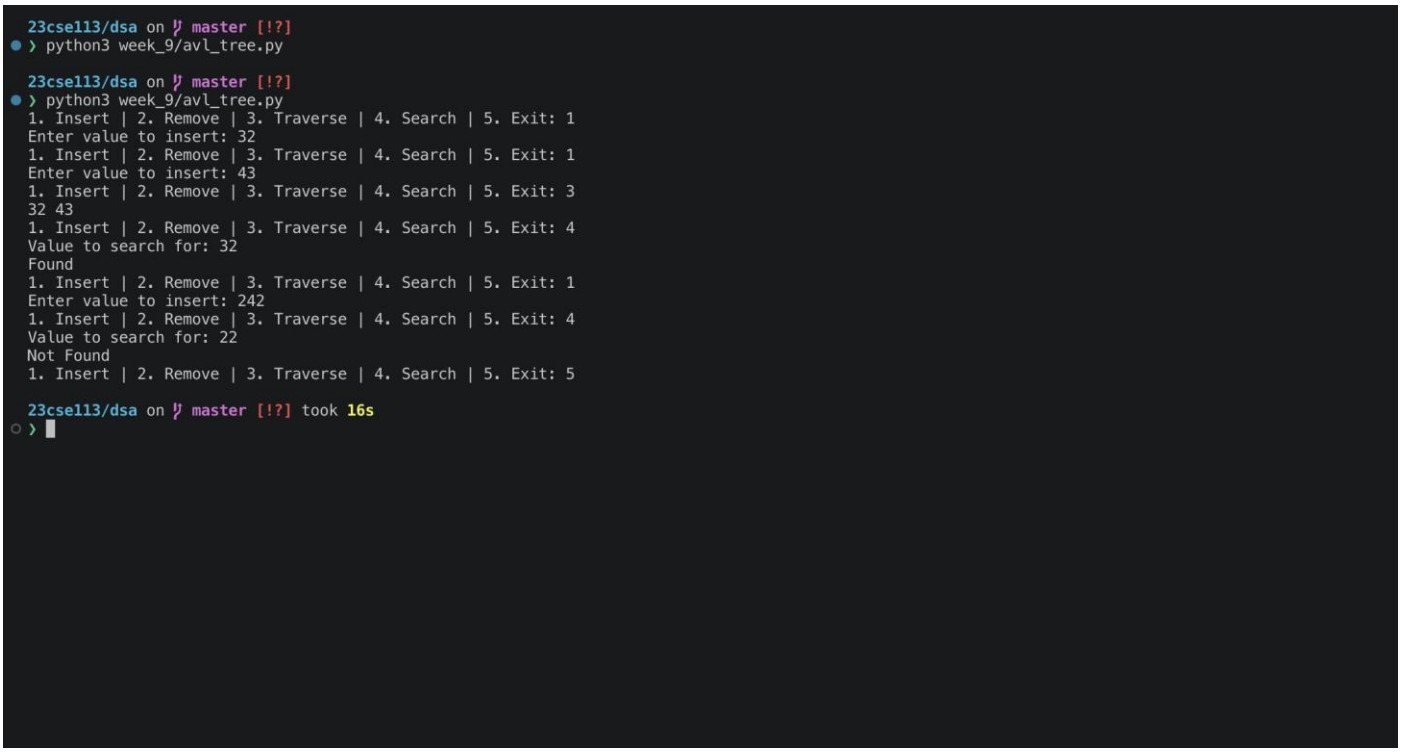
elif x == '3':
tree.inorder(root)
print()

elif x == '4':
value = int(input("Value to search for: "))
print("Found" if tree.search(root, value) else "Not Found")
```

```
elif x == '5':
break

else:
print("Invalid choice")
```

Output:



Exmplanation:

Explanation: The following code demonstrates an AVL Tree in Python. An AVL Tree is a self-balancing Binary Search Tree (BST) in which the difference between the heights of the left and right subtrees of every node is at most 1. This difference is called the balance factor. Whenever insertion or deletion makes the tree unbalanced, rotations are performed to restore balance. This helps maintain efficient searching, insertion, and deletion operations.

Node class: This class represents a single node in the AVL Tree. The __init__() function initializes the node with a value, sets its left and right children to None, and initializes its height to 1 because a newly created node is a leaf node.

height(): This function returns the height of a node. If the node is None, it returns 0. Otherwise, it returns the height stored in the node.

balance(): This function calculates the balance factor of a node by subtracting the height of its right subtree from the height of its left subtree. A balance factor of -1, 0, or 1 indicates that the node is balanced. A value greater than 1 or less than -1 indicates that rebalancing is required.

right_rotate(): This function performs a right rotation to balance a subtree that is left-heavy. The left child becomes the new root of the subtree, the original root becomes its right child, and the left child's right subtree is transferred to the original root's left side. The heights of the affected nodes are then updated.

left_rotate(): This function performs a left rotation to balance a subtree that is right-heavy. The right child becomes the new root of the subtree, the original root becomes its left child, and the right child's left subtree is transferred to the original root's right side. The heights of the affected nodes are then updated.

insert(): This function inserts a new value into the AVL Tree while maintaining the Binary Search Tree property. If the current node is None, a new node is created. If the value is smaller than the current node's value, it is inserted into the left subtree. If it is greater, it is inserted into the right subtree. Duplicate values are ignored. After insertion, the node's height and balance factor are updated. If the node becomes unbalanced, one of the following four cases is handled:

1. LL (Left-Left) Case: Occurs when a node becomes unbalanced due to insertion into the left subtree of its left child. It is corrected using a right rotation.
2. RR (Right-Right) Case: Occurs when a node becomes unbalanced due to insertion into the right subtree of its right child. It is corrected using a left rotation.
3. LR (Left-Right) Case: Occurs when a node becomes unbalanced due to insertion into the right subtree of its left child. It is corrected using a left rotation on the left child, followed by a right rotation on the node.
4. RL (Right-Left) Case: Occurs when a node becomes unbalanced due to insertion into the left subtree of its right child. It is corrected using a right rotation on the right child, followed by a left rotation on the node.

The function returns the updated root of the subtree after insertion and any required rotations.

min_value_node(): This function finds the node containing the smallest value in a subtree. It repeatedly moves to the left child until it reaches the leftmost node. This is used during deletion when a node has two children.

delete(): This function removes a specified value from the AVL Tree. If the value is smaller or greater than the current node's value, it recursively searches the left or right subtree. When the node is found, there are three cases:

1. No left child: The node is replaced by its right child.
2. No right child: The node is replaced by its left child.
3. Two children: The smallest value in the right subtree is found using min_value_node(). Its value replaces the deleted node's value, and the original node containing that value is removed.

After deletion, the function updates the node's height and balance factor. If the node becomes unbalanced, it performs the appropriate LL, LR, RR, or RL rotations to restore the AVL Tree property. The updated subtree root is then returned.

search(): This function searches for a specified value in the AVL Tree. If the current node is None or contains the required value, it returns that node. If the value is smaller than the current node's value, the

search continues in the left subtree; otherwise, it continues in the right subtree. If the value does not exist, the function returns None.

inorder(): This function performs an inorder traversal of the AVL Tree. It recursively visits the left subtree, prints the current node's value, and then visits the right subtree. Since the AVL Tree follows the Binary Search Tree property, the values are displayed in ascending order.

Week 10

Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).

```
class HashTable:
def __init__(self, size, method):
self.size = size
self.method = method
self.table = [[] for _ in range(size)] if method == 1 else [None] * size

def hash(self, key):
return key % self.size

def insert(self, key):
index = self.hash(key)

if self.method == 1: # Separate Chaining
if key not in self.table[index]:
self.table[index].append(key)

elif self.method == 2: # Linear Probing
for i in range(self.size):
pos = (index + i) % self.size
if self.table[pos] is None:
self.table[pos] = key
return
print("Hash table is full!")

elif self.method == 3: # Quadratic Probing
for i in range(self.size):
pos = (index + i * i) % self.size
if self.table[pos] is None:
self.table[pos] = key
return
print("No empty position found!")

def search(self, key):
index = self.hash(key)

if self.method == 1:
return key in self.table[index]

for i in range(self.size):
if self.method == 2:
pos = (index + i) % self.size
else:
pos = (index + i * i) % self.size

if self.table[pos] is None:
return False

if self.table[pos] == key:
return True

return False
```

```
def delete(self, key):
index = self.hash(key)

if self.method == 1:
if key in self.table[index]:
self.table[index].remove(key)
print("Deleted successfully!")
else:
print("Key not found!")
return

for i in range(self.size):
if self.method == 2:
pos = (index + i) % self.size
else:
pos = (index + i * i) % self.size

if self.table[pos] is None:
break

if self.table[pos] == key:
self.table[pos] = "DELETED"
print("Deleted successfully!")
return

print("Key not found!")

def display(self):
for i in range(self.size):
print(i, ":", self.table[i])



def main():
print("HASHING TECHNIQUES")
print("1. Separate Chaining")
print("2. Linear Probing")
print("3. Quadratic Probing")

method = int(input("Choose hashing technique: "))
size = int(input("Enter hash table size: "))

if method not in [1, 2, 3] or size <= 0:
print("Invalid choice or size!")
return

h = HashTable(size, method)

while True:
print("\n--- MENU ---")
print("1. Insert")
print("2. Search")
print("3. Delete")
print("4. Display")
print("5. Exit")

choice = int(input("Enter choice: "))
```

```
if choice == 1:
key = int(input("Enter key: "))
h.insert(key)

elif choice == 2:
key = int(input("Enter key: "))
if h.search(key):
print("Key found!")
else:
print("Key not found!")

elif choice == 3:
key = int(input("Enter key: "))
h.delete(key)

elif choice == 4:
h.display()

elif choice == 5:
print("Exiting...")
break

else:
print("Invalid choice!")



main()
```

Output:

```
23cse113/dsa/week_10 on  master [!?] via  v3.14.6
❯ python3 hashing.py
HASHING TECHNIQUES
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
Choose hashing technique: 3
Enter hash table size: 5

--- MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter key: 2342

--- MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 4
0 : None
1 : None
2 : 2342
3 : None
4 : None

--- MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 5
Exiting...

23cse113/dsa/week_10 on  master [!?] via  v3.14.6 took 9s
❯
```

Explanation: The following code demonstrates the implementation of a Hash Table in Python using three different collision handling techniques: Separate Chaining, Linear Probing, and Quadratic Probing. A Hash Table is a data structure that stores keys using a hash function to calculate their positions (indices) in an array. The hash function used in this program is key % size, which determines the index where a key should be stored. When two or more keys produce the same index, a collision occurs. The program handles collisions using the selected hashing technique.

HashTable class: This class implements the hash table and provides functions for insertion, searching, deletion, and displaying elements.

init(): This function initializes the hash table with a given size and hashing method. It stores the size and method as instance variables. If the selected method is 1 (Separate Chaining), it creates a list of empty lists, where each index can store multiple keys. Otherwise, it creates a list of the specified size filled with None, where each position can store a single key.

hash(): This function calculates the hash index for a given key using the modulo operation (key % self.size). The resulting index determines the initial position where the key should be inserted or searched for in the hash table.

insert(): This function inserts a key into the hash table. First, it calculates the index using the hash function. Depending on the selected hashing method, insertion is performed differently:

1. Separate Chaining: If the key is not already present in the list at the calculated index, it is appended to that list. Multiple keys can be stored at the same index using a linked-list-like structure (implemented here using Python lists).

2. Linear Probing: If the calculated index is occupied, the function checks the next positions sequentially using (index + i) % size until an empty position is found. The key is inserted into the first available position. If the table is full, it displays "Hash table is full!".
3. Quadratic Probing: If the calculated index is occupied, the function checks alternative positions using (index + i * i) % size. The square of the iteration number is added to the original index to find the next position. If an empty position is found, the key is inserted. Otherwise, it displays "No empty position found!".

search(): This function searches for a given key in the hash table. First, it calculates the initial index using the hash function.

- Separate Chaining: It checks whether the key exists in the list at the calculated index.
- Linear Probing: It checks positions sequentially using (index + i) % size.
- Quadratic Probing: It checks positions using (index + i * i) % size.

If the key is found, the function returns True. If an empty position (None) is encountered before finding the key, it returns False. If the key is not found after checking all possible positions, it also returns False.

delete(): This function removes a specified key from the hash table.

- Separate Chaining: It checks whether the key exists in the list at the calculated index. If found, it removes the key using the remove() function. Otherwise, it displays "Key not found!".
- Linear Probing: It searches for the key using sequential probing. If found, its position is replaced with the string "DELETED".
- Quadratic Probing: It searches for the key using quadratic probing. If found, its position is replaced with "DELETED".

The "DELETED" marker is used instead of None because replacing a deleted element with None could interrupt the probing sequence and cause subsequent searches to fail. If the key is not found, the function displays "Key not found!".

display(): This function displays the contents of the hash table. It iterates through every index and prints the index along with the corresponding stored value. In Separate Chaining, it displays the list of keys stored at each index, while in Linear and Quadratic Probing, it displays individual keys or empty positions.

main(): This function acts as the main driver of the program. It first displays the three available hashing techniques and asks the user to select one. It then asks for the hash table size. If the selected method is invalid or the size is less than or equal to zero, the program displays an error message and terminates.

After creating a HashTable object, it repeatedly displays a menu with the following operations:

1. Insert: Accepts a key from the user and inserts it into the hash table.
2. Search: Accepts a key and checks whether it exists in the hash table. It displays "Key found!" or "Key not found!".
3. Delete: Accepts a key and removes it from the hash table.
4. Display: Displays all indices and their corresponding values.

5. Exit: Terminates the program.